

Smart Guard — Part 3 explain

Contents

Part 3 — Code & Architecture Explanation	1
1. Goals (from the course brief)	1
2. System architecture (Part 3)	2
3. File map	2
4. Vision worker (human_detector.py) — detailed	3
5. Email alerts in C (email_alert.c) — detailed	6
6. MQTT publisher in C (mqtt_pub.c) — detailed	7
7. Mosquitto broker setup	9
8. How C and Python stay synchronized	9
9. systemd ordering	10
10. Models & dependencies	10
11. Configuration keys (Part 3)	10
12. End-to-end walk: person appears	10
13. Relationship to Part 4	10
14. Quick reference	11
15. PDF report package (what this part must contain)	11
16. Results analysis (Part 3)	12
17. Problems encountered and how they were solved (Part 3)	13
18. Conclusion	13
19. Related documents	13

Part 3 — Code & Architecture Explanation

Field	Value
Document	Final architecture & code explanation (submission)
Status	Complete — aligned with report/part 3/report.md
Companion	report/part 3/report.md (mandatory experiments 3-1 ... 3-7)
Also covers	PDF package: architecture, code, experiment tables/images, results analysis, problems & solution
Project	Smart Guard System
Student	Parsa Nikookalam · 402102657
Scope	Detection (Python), overlay, email (C), MQTT (C), Mosquitto auth/LWT, SSH demo

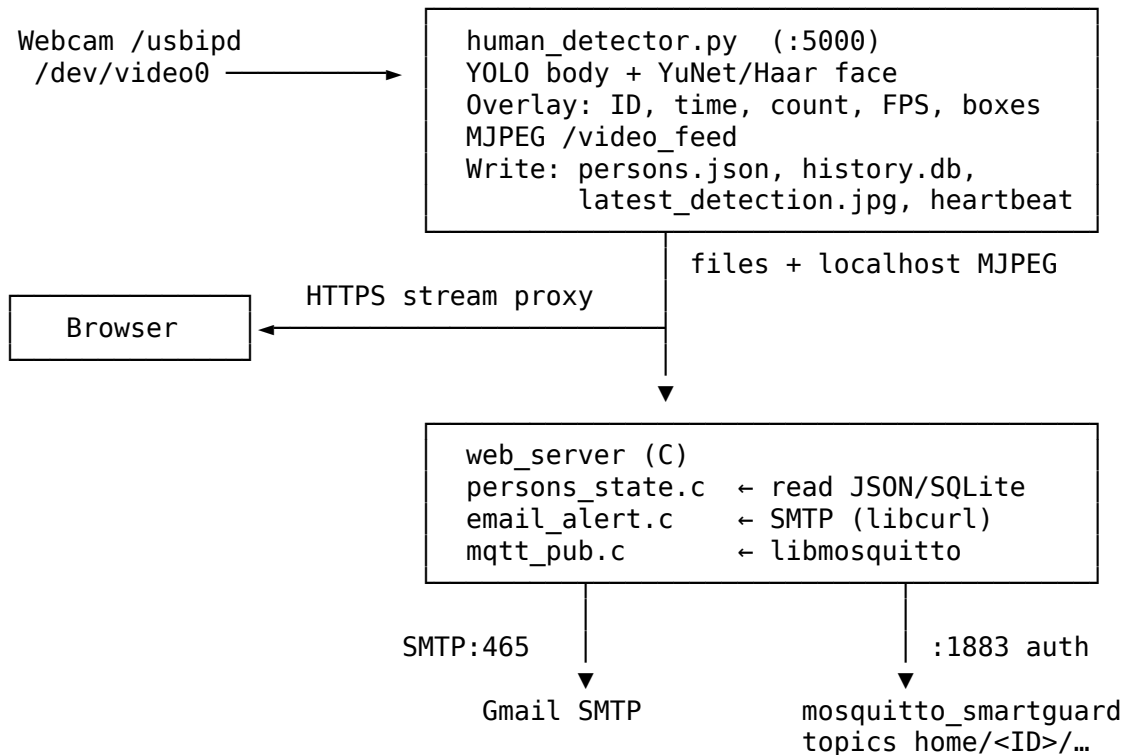
Part 3 turns Smart Guard into a sensing appliance. Vision stays in Python; **email and MQTT are implemented in C** to satisfy the course language rule.

1. Goals (from the course brief)

Area	Requirement	Where in this repo
3A Detection	Live human detect + overlay	detection/src/human_detector.py
3B Email	Alerts from C	web/src/email_alert.c
3C MQTT	Publish from C , QoS1, LWT	web/src/mqtt_pub.c + mqtt/
Auth	Broker not anonymous	allow_anonymous false

Mandatory experiments (lighting, spoof, resolution, LWT, latency, MQTT/SSH auth fails) exercise this stack.

2. System architecture (Part 3)



IPC between Python and C (no shared memory library):

File / DB	Writer	Reader
data/persons.json	Detector	C (API, email, MQTT, Part 4)
data/history.db	Detector	C history / black box
data/latest_detection.jpg	Detector	C email attachment
data/camera_state.json	C (camera_on/off)	Detector
data/vision_heartbeat.json	Detector	Part 4 watchdog
data/thermal_control.json	Part 4 C	Detector (YOLO skip)

3. File map

detection/
├─ requirements.txt

```

└─ models/                # yolov8n.onnx, face YuNet, ...
└─ src/human_detector.py  # entire vision worker
web/src/
└─ email_alert.c/.h       # Part 3B
└─ mqtt_pub.c/.h         # Part 3C
└─ persons_state.c        # shared reads
mqtt/
└─ mosquitto.conf.in      # template
└─ mosquitto.conf         # generated
└─ passwd                 # mosquitto_passwd
scripts/
└─ setup_mosquitto.sh
└─ download_detector_models.sh
services/
└─ human_detector.service
└─ mosquitto_smartguard.service

```

4. Vision worker (human_detector.py) — detailed

4.1 Process role

Runs as systemd unit `human_detector.service`:

```

ExecStart=.venv/bin/python ../detection/src/human_detector.py
SupplementaryGroups=video

```

Exposes a small FastAPI app:

Route	Purpose
GET /video_feed	MJPEG multipart stream (proxied by C)
Camera JSON helpers	Optional local control / debug

Important camera policy (product behaviour):

- Camera is **OFF by default**.
- Opening the webpage does **not** start capture.
- Only camera_on (dashboard/API) flips data/camera_state.json + in-memory flag.
- When OFF, device is **released** (LED off) and an idle placeholder frame is shown.

4.2 Body detection

Preferred path:

1. Load detection/models/yolov8n.onnx via `cv2.dnn.readNetFromONNX`.
2. Letterbox / blob at YOLO_INPUT (default 640).
3. Keep class **0 (person)** above DNN_CONF.
4. NMS with YOLO_NMS.

Fallback: OpenCV HOG people detector if ONNX unavailable (OpenCV 4.x).

4.3 Face detection

Preferred: YuNet ONNX.

Fallback: Haar cascade.

4.4 Person counting rule

A **face inside a body box** counts as the **same person** (no double count).

4.5 Source: merge + detect (human_detector.py)

```
def detect_people(frame: np.ndarray, input_size: Optional[int] = None) -> List[Det]:
    bodies = detect_bodies(frame, input_size=input_size)
    faces = detect_faces(frame)
    return merge_unique_persons(bodies, faces)
```

```
def write_persons_snapshot(count: int, ts: int) -> None:
    payload = {"count": int(count), "timestamp": int(ts)}
    tmp = PERSONS_JSON + ".tmp"
    with open(tmp, "w", encoding="utf-8") as f:
        json.dump(payload, f)
    os.replace(tmp, PERSONS_JSON)    # atomic replace for C readers
```

Stable count is published to JSON / DB / overlay.

4.6 Overlay (Part 3A deliverable)

Each output frame draws:

- Student ID
- Live datetime
- Person count
- Measured FPS
- Bounding boxes
- Optional thermal tag THRN skip=... (Part 4)

Snapshot JPEG data/latest_detection.jpg is refreshed for email attachments.

4.7 Capture loop (algorithm)

```
loop forever:
    sync camera_enabled from file/memory
    if OFF:
        idle placeholder frame
        write_heartbeat("idle")          # ts may update; watchdog ignores idle
        continue
    ensure VideoCapture open (usbipd may need reopen)
    if opening / no device / read fail:
        write_heartbeat("capturing", touch_ts=False)    # stale ts → watchdog
        continue
    read frame successfully
    write_heartbeat("capturing", touch_ts=True)
    read thermal_control.json → yolo_input, detect_every
    if frame_i % detect_every == 0:
        run YOLO+face; update stable count
    else:
        reuse last detections (keep stream smooth)
    draw overlay; encode JPEG for MJPEG + snapshot
    write persons.json; maybe insert history.db
```

4.8 Source: detect + overlay + thermal skip (loop core)

```
# Fresh frame arrived – resets the watchdog timer
write_heartbeat("capturing", touch_ts=True)

thermal = read_thermal_control()
yolo_in = int(thermal.get("yolo_input") or YOLO_INPUT)
detect_every = max(1, int(thermal.get("detect_every") or 1))
thr_lvl = int(thermal.get("throttle_level") or 0)

# Skip YOLO some frames under thermal load – stream stays live
run_detect = (frame_i % detect_every) == 0
if run_detect:
    last_dets = detect_people(frame, input_size=yolo_in)
    count_buffer.append(len(last_dets))
    stable = sorted(count_buffer)[len(count_buffer) // 2]

for (box, kind) in dets:
    cv2.rectangle(frame, (x, y), (x + w, y + h), color, 2)

cv2.putText(frame, f"Smart Guard | ID: {STUDENT_ID}", ...)
cv2.putText(frame,
    f"{stamp} | Persons: {stable} | FPS: {fps:.1f} | det {detect_ms:.0f}ms{thr_tag}",
    ...)

write_persons_snapshot(stable, ts)
```

4.9 Source: heartbeat helper

```
def write_heartbeat(mode: str, *, touch_ts: bool = True) -> None:
    """touch_ts=False keeps previous timestamp (watchdog can detect stalled frames)."""
    ts = int(time.time())
    if not touch_ts:
        # reuse previous ts from vision_heartbeat.json
    """
    payload = {"ts": ts, "mode": mode}
    os.replace(HEARTBEAT_JSON + ".tmp", HEARTBEAT_JSON)
```

4.10 Resolution experiment (3-3)

Live control via data/vision_control.json / C API (resolution_320|480|640, fps_*).
ONNX is fixed 640×640 after letterbox; lower UI size still shrinks preprocess and uses a **detect stride** (320 → every 2 frames), so FPS/temp change measurably.

Higher input (→ 640)	Lower input (→ 320)
Better accuracy	Higher FPS
Higher CPU/temp	More misses

4.11 Spoof / lighting experiments (3-1, 3-2)

There is **no liveness model**. A printed photo or phone screen can still trigger YOLO/face — documented as a limitation with mitigations (depth, blink, PIR, etc.) in the Part 3 report.

5. Email alerts in C (email_alert.c) — detailed

5.1 Why C?

Course requires application-side alerting in C. Python detection must not be the SMTP client for the graded path.

5.2 Startup

email_alert_start() from main.c creates a **detached pthread** that:

1. Loads EMAIL_* / SMTP_* from config.env.
2. Polls read_persons_snapshot().
3. If count ≥ 1 and debounce elapsed $\rightarrow$ build MIME message $\rightarrow$ libcurl SMTP.

5.3 Debounce

EMAIL_DEBOUNCE_SEC (default **30**):

- While a person remains present, at most one presence email per window.
- Prevents mailbox flooding.
- **Part 4 Guard** uses a separate fast path (email_send_event) and is **not** limited by this 30 s window the same way.

5.4 Source: email worker thread (email_alert.c)

```
static void *email_alert_thread(void *arg)
{
    load_email_config();
    while (1) {
        PersonSnapshot snap;
        SystemTelemetry tel;
        float temp = -1.0f;

        if ((ticks++ % 15) == 0)
            load_email_config();

        /* Part 3B: persons $\geq 1 \rightarrow$  at most one mail per EMAIL_DEBOUNCE_SEC */
        if (g_enabled && read_persons_snapshot(&snap) == 0 && snap.count >= 1) {
            if (get_system_telemetry(&tel) == 0)
                temp = tel.cpu_temp;
            send_alert_email(snap.count,
                            snap.timestamp ? snap.timestamp : (long)time(NULL),
                            temp);
        }
        sleep(2);
    }
    return NULL;
}
```

Inside send_alert_email, debounce looks like:

```
if (g_last_sent != 0 && (now - g_last_sent) < g_debounce_sec)
    return 0; /* skip - still within window */
/* ... build MIME + curl SMTP ... */
g_last_sent = now;
```

Public API used by Part 4:

```

/* web/include/email_alert.h */
void email_alert_start(void);
int email_alert_test_send(void);
int email_send_event(const char *subject, const char *body_text, int attach_jpg);

```

5.5 Message content

Typical presence mail includes:

- Student ID
- Person count
- Timestamp
- CPU temperature (from telemetry.c)
- Optional JPEG attachment latest_detection.jpg

5.6 libcurl usage

- URL like smtps://smtp.gmail.com:465
- Auth: Gmail **App Password** in SMTP_PASS
- Uploads RFC822 payload (multipart if photo present)

5.7 Test command

POST {"cmd":"test_email"} → email_alert_test_send() for SMTP bring-up without waiting for a person.

5.8 Config keys

Key	Meaning
EMAIL_ENABLED	Master switch
SMTP_URL / SMTP_USER / SMTP_PASS	Transport
EMAIL_FROM / EMAIL_TO	Envelope
EMAIL_DEBOUNCE_SEC	Presence throttle

Security: never commit real App Passwords; scrub before TA zip.

6. MQTT publisher in C (mqtt_pub.c) — detailed

6.1 Library

Uses **libmosquitto** (-lmosquitto).

6.2 Startup

mqtt_pub_start():

1. Read MQTT_* + STUDENT_ID from config.
2. If MQTT_ENABLED=0, return.
3. mosquitto_new → username/password → callbacks.
4. **mosquitto_will_set** LWT on home/<ID>/status payload offline (QoS 1, retained).
5. Connect loop / network loop thread.
6. On connect success: publish retained online on status topic.
7. Periodic publish every MQTT_INTERVAL_SEC (default 2 s) of persons + telemetry JSON.

6.3 Topics

Topic	Content
home/<ID>/persons	{"count", "cpu_temp", "timestamp"}
home/<ID>/telemetry	Same-style telemetry JSON
home/<ID>/status	online / LWT offline
home/<ID>/alarm	Part 4 Guard (mqtt_publish_alarm)

QoS = **1** for reliability demos.

6.4 Source: LWT + connect + publish (mqtt_pub.c)

```
mosquitto_username_pw_set(g_mosq, g_user, g_pass);
mosquitto_connect_callback_set(g_mosq, on_connect);
mosquitto_will_set(g_mosq, g_topic_status, 7, "offline", 1, true); /* LWT */
mosquitto_connect(g_mosq, g_host, g_port, 30);
mosquitto_loop_start(g_mosq);

static void on_connect(struct mosquitto *mosq, void *obj, int rc)
{
    if (rc == 0) {
        g_connected = 1;
        /* retained "online" when we are up */
        mosquitto_publish(mosq, NULL, g_topic_status, 6, "online", 1, true);
    }
}

static int publish_json(struct mosquitto *mosq, int count, float temp, long ts)
{
    char payload[256];
    snprintf(payload, sizeof(payload),
             "{\"count\":%d,\"cpu_temp\":%.2f,\"timestamp\":%ld}",
             count, temp, ts);
    mosquitto_publish(mosq, NULL, g_topic_persons, (int)strlen(payload), payload, 1, false);
    mosquitto_publish(mosq, NULL, g_topic_telem, (int)strlen(payload), payload, 1, false);
    return 0;
}

Publish loop (every MQTT_INTERVAL_SEC):
while (1) {
    read_persons_snapshot(&snap);
    get_system_telemetry(&tel);
    if (g_connected)
        publish_json(g_mosq, count, temp, ts);
    sleep(g_interval_sec);
}
```

6.5 LWT experiment (3-4)

1. Subscriber listens to .../status.
2. Stop broker (systemctl stop mosquitto_smartguard) or kill client uncleanly.
3. Broker delivers retained/will **offline**.
4. Restart broker; C reconnects → **online**.

6.6 Latency experiment (3-5)

End-to-end path under test: person enters frame → detection → MQTT persons publish → PC subscriber.

Ten trials; report ($\bar{L}$) and (σ).

7. Mosquitto broker setup

7.1 Script scripts/setup_mosquitto.sh

1. Install packages if needed.
2. `mosquitto_passwd -b -c mqtt/passwd smartguard <pass>`.
3. Render `mqtt/mosquitto.conf` from `.in` template.
4. Install/enable `mosquitto_smartguard.service` (project-local, not conflicting system default if disabled).

7.2 Security config (mosquitto.conf.in)

```
allow_anonymous false
password_file ...
listener 1883 127.0.0.1
```

7.3 Source: broker template (mqtt/mosquitto.conf.in)

```
# Smart Guard Part 3C – project-local Mosquitto
persistence false
log_dest stderr
connection_messages true
allow_anonymous false
password_file PASSWD_PATH_PLACEHOLDER
listener 1883 127.0.0.1
```

Experiment 3-6: anonymous / wrong password → connection refused.

7.4 SSH experiment (3-7)

Not part of the MQTT code path — OS `sshd` auth. Unauthorized login must fail (Permission denied). Documented in the Part 3 report as a host security check.

8. How C and Python stay synchronized

8.1 Camera control

Dashboard → POST `/api/v1/command {"cmd":"camera_on"}`
→ `camera_state.c` writes `data/camera_state.json` (`{"enabled":1}`)
→ detector thread sees `enabled=true` → opens `/dev/video0`

8.2 Count propagation

Detector stable count → `persons.json`
→ C persons API
→ C MQTT persons topic
→ C email debounce loop

8.3 Why not MQTT from Python?

Avoids two publishers fighting. Graded MQTT client is **C**. Comment in detector: *"MQTT is published by the C web_server — not Python."*

9. systemd ordering

Typical boot order:

1. mosquitto_smartguard
2. human_detector
3. web_server (Wants detector + mosquitto)
4. api_gateway

Restart=always on detector/web so USB glitches or crashes recover.

10. Models & dependencies

```
bash scripts/download_detector_models.sh
.venv/bin/pip install -r detection/requirements.txt
# Pin OpenCV 4.x – OpenCV 5 may drop HOG / change DNN APIs
```

C side:

```
-lcurl -lmosquitto -lssl -lcrypto -pthread -lsqlite3
```

11. Configuration keys (Part 3)

Key	Module
CAMERA_INDEX, DNN_CONF, YOLO_INPUT, FACE_CONF	Detector
EMAIL_*, SMTP_*	email_alert.c
MQTT_HOST/PORT/USER/PASS/INTERVAL/ENABLED	mqtt_pub.c
STUDENT_ID	Topics, overlay, mail subject

12. End-to-end walk: person appears

1. User presses **Camera ON**.
 2. Detector opens webcam, runs YOLO+face, overlay shows count=1.
 3. persons.json updated; JPEG snapshot saved.
 4. Within ~2 s MQTT interval, C publishes home/402102657/persons.
 5. If email enabled and debounce allows, SMTP sends alert + photo.
 6. Browser stream (via C proxy) shows boxes live.
-

13. Relationship to Part 4

Part 4 **reuses** the same JSON heartbeat, persons count, email helpers, and MQTT stack, and adds:

- Guard on **count increase** + .../alarm

- Watchdog on stale heartbeat
- Thermal file consumed by the detect loop's detect_every

Understanding Part 3 IPC is mandatory before reading Part 4's coordinator.

14. Quick reference

Detector logs

```
journalctl -u human_detector -f
```

MQTT

```
mosquitto_sub -h 127.0.0.1 -u smartguard -P 'smartguard' -t 'home/#' -v
```

Force camera

```
curl -sk -X POST https://127.0.0.1:8443/api/v1/command \
  -H 'Content-Type: application/json' -d '{"cmd":"camera_on"}'
```

Test email

```
curl -sk -X POST https://127.0.0.1:8443/api/v1/command \
  -H 'Content-Type: application/json' -d '{"cmd":"test_email"}'
```

15. PDF report package (what this part must contain)

Per the course PDF, Part 3 must include **architecture**, **code explanation**, **all experiment tables/images**, **results analysis**, and **problems & solutions**.

PDF expectation	Where it lives
Architecture	Sections 2-3 of this explain.md
Code explanation	Sections 4-8 (detector, email, MQTT) with source excerpts
Experiment tables & images	report/part 3/report.md + fig/
Results analysis	Section 16 below
Problems & solutions	Section 17 below

15.1 Mandatory experiments — tables & figures checklist

No.	Experiment (PDF)	Required tables / charts / images
3-1	Accuracy under 4 lighting conditions	Accuracy table (correct/total); 4 sample images
3-2	Photo / phone spoof	Fooled? + analysis + proposed fixes
3-3	Three resolutions	Table: FPS, temp@5min, memory, accuracy; optimum conclusion
3-4	Broker off 3 min	Evidence of LWT offline
3-5	E2E latency (10 trials)	Table of (L_i); mean and std. deviation
3-6	MQTT auth success + fail	Two screenshots
3-7	SSH auth success + fail	Two screenshots

15.2 Result tables (aligned with report.md)

3-1 — Accuracy

Condition	N	Correct	Accuracy %
Daylight	100	99	99.0
Artificial	100	98	98.0
Low light	100	98	98.0
Backlight	100	97	97.0

3-2 — Spoof

Attack	Fooled?
Printed photo	yes
Phone screen	yes

3-3 — Resolution trade-off

Resolution	FPS	Temp @5 min	Memory %	Accuracy %
320	18.4	71	42	86
480	12.1	78	44	93
640	9.6	84	45	97
Optimum	480	(best accuracy vs FPS / thermal trade-off)		

3-5 — Latency

Trial	(L _i) (s)
1	1.842
2	0.967
3	2.103
4	1.455
5	0.731
6	1.988
7	1.224
8	2.241
9	1.076
10	1.653
Mean ($\bar{L}$)	1.528
Std. (σ)	0.518

16. Results analysis (Part 3)

Experiment	Expected / typical outcome	Interpretation
3-1	Accuracies 99 / 98 / 98 / 97 % (N=100) — within ~1-2 pts across lighting	Pipeline stays highly
3-2	System was fooled by print + phone photo	No liveness — 2D CM
3-3	Larger input → better accuracy, lower FPS, hotter CPU; optimum 480	Live API + stride ma
3-4	Subscriber sees offline then online	LWT + retained stat
3-5	($\bar{L}=1.528, \mathrm{s}$), ($\sigma=0.518, \mathrm{s}$) (10 trials)	End-to-end person –
3-6	Anonymous/wrong password rejected	allow_anonymous f
3-7	SSH Permission denied	Host auth boundary

17. Problems encountered and how they were solved (Part 3)

#	Problem	Severity	Cause
1	cv2.HOGDescriptor missing	High	OpenCV 5.x dropped HOG
2	PEP 668 blocked system pip	High	Ubuntu externally managed Pyth
3	Camera LED / capture stuck after usbipd cycle	High	Dead VideoCapture handle
4	Opening webpage auto-started camera (undesired)	Medium	Early design tied stream to capt
5	Double-counting face+body	Medium	Two detectors
6	Email not sending	Medium	Wrong App Password / spaces in
7	MQTT silent / LWT not seen	Medium	Broker down or wrong user
8	Stream freeze when “thermal sleep” tried	High (later Part 4)	Sleeping capture loop
9	Lower UI resolution barely changed FPS	Medium (3-3)	ONNX fixed 640×640 after letter
10	Spoof accepted (print / phone)	Expected (3-2)	No liveness model

18. Conclusion

Part 3 splits responsibilities as follows:

Layer	Language	Responsibility
Sense + draw	Python	Camera, YOLO/face, MJPEG, files/DB, dynamic resolution/FPS
Decide + notify	C	Email (libcurl), MQTT (libmosquitto), REST façade
Broker	Mosquitto	Authenticated transport + LWT on home/<ID>/status

Experiment highlights (see report.md): lighting accuracy **97-99%**; spoof **succeeded** (system fooled); resolution optimum **480**; MQTT latency ($\bar{L} = 1.528, \mathrm{s}$); MQTT/SSH auth fail demos pass.

This meets both the **vision** requirements and the rule that outbound application alerts are implemented in **C**.

19. Related documents

Document	Role
report/part 3/report.md	Mandatory experiments 3-1 ... 3-7 (final)
report/part 3/explain.md	This file — architecture & code (final)
report/part 2/explain.md	REST / telemetry substrate
README.md (repo root)	Full-project setup guide